Using instance_eval and class_eval

No matter where you are in a Ruby program, self always has a value determined by your location in the code. Sometimes, though, it's really useful to be able to manage that relationship and change the value of self for a while.

The methods instance_eval, class_eval, and module_eval let you set self to be an arbitrary object, evaluate the code in a block with that object as self, and then reset self:

```
"cat".instance_eval do
  puts "Upper case = #{upcase}"
  puts "Length is #{self.length}"
end
```

produces:

```
Upper case = CAT
Length is 3
```

Inside the instance_eval block, self evaluates as the object that received the instance_eval message.

All the _eval methods take a string instead of a block argument, where the string is evaluated by Ruby with their receiver as self. However, the string version is considered dangerous, and you shouldn't use it.

First, it is slow—calling eval effectively compiles the code in the string before executing it. But, even worse, eval from a string can be dangerous. If there's any chance that external data—stuff that comes from outside your application—can wind up inside the string argument to eval, then you have a security hole, because that external data may end up containing arbitrary code that your application will blindly execute.

class_eval and instance_eval both set self for the duration of the block. However, they differ in the way they set up the environment for method definition. This can make a big difference when the receiver is a class. The method class_eval sets things up as if you were in the body of a class definition, so when the receiver is a class, method definitions will define instance methods in that class:

```
class MyClass
end

MyClass.class_eval do
  def instance_method
    puts "In an instance method"
  end
end

obj = MyClass.new
obj.instance_method
```

produces:

```
In an instance method
```

In contrast, calling instance_eval acts as if you were working inside the singleton class of self. Therefore, if the receiver is a class any methods you define will become singleton methods of that class, which is to say that they become class methods:

```
class MyClass
end
```